

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

«Программирование»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Строки и ввод-вывод.

Выполнил:

Студент группы N3150

Пахомов Владимир Юрьевич



Проверил:

Гирик А.В.

Санкт-Петербург

2024г.

Оглавление

Задание. Вариант 8-1-1-2.....	3
Make-файл.....	3
Примеры работы программы на различных исходных данных.....	4
Исходный текст программы с комментариями.....	6

Задание. Вариант 8-1-1-2

Таблица 1. Данные для поиска:

8. Дата и время в формате HH:MM:SS dd.mm.yyyy

Примеры: 12:24:01 01.02.2001, 00:00:00 31.12.2023

Таблица 2. Опции, с помощью которых задаётся ограничение преобразования:

-b=M

Опция -b задаёт номер начальной строки (строки нумеруются с единицы).

Преобразование выполняется, начиная со строки M. Если опция -b не указана, то преобразование начинается с первой строки.

-e=N

Опция -e задаёт номер конечной строки. Преобразование выполняется до строки N включительно. Если опция -e не указана, преобразование продолжается до последней строки.

Таблица 3. Разметка Markdown, используемая для выделения найденных фрагментов текста:

Курсив **Пример**

Таблица 4. Цвет для выделения найденных фрагментов текста при выводе с опцией -c:

Зелёный (green) 32

Make-файл

.PHONY: all clean

APP=prg3vypN3150

CFLAGS=-Wall -Wextra -Werror -g

all: \$(APP)

\$(APP): \$(APP).c

gcc -o \$(APP) \$(CFLAGS) \$(APP).c

clean:

rm \$(APP)

Примеры работы программы на различных исходных данных

Без опций и указания входных и выходных потоков программа обрабатывает строку из ввода пользователя и выводит результат в консоль:

```
~ /C_Workspace/prg3vypN3150 ./prg3vypN3150
Hello, World! Now 19:11:46 26.12.2024
time on different lines: 19:1
1:46
date on different lines: 26.
12.2024
wrong formats: 25:12:00 1:15:18 45.14.2999 1.12.1890

Hello, World! Now *19:11:46* *26.12.2024*
time on different lines: *19:1
1:46*
date on different lines: *26.
12.2024*
wrong formats: 25:12:00 1:15:18 45.14.2999 1.12.1890
```

При использовании опции -v выводится информация об авторе:

```
~ /C_Workspace/prg3vypN3150 ./prg3vypN3150 -v
Пахомов Владимир Юрьевич, гр. N3150
Вариант: 8-1-1-2
```

При добавлении опций, входного и выходного потока программа обрабатывает переданные аргументы и меняет своё поведение:

```
~ /C_Workspace/prg3vypN3150 ./prg3vypN3150 -b=2 -e=10 in.txt

Hello World 17:25:010 10.10.2000
*00:00:00*
99:99:99
0:00:0
*12:12:12*
*14:13:14*
*17:17:17*
*21.12.2002*
*00:
00:00* *00.00.9999*

10.
01.20
00
15.01.2022
```

```
~ /C_Workspace/prg3vypN3150 ./prg3vypN3150 -n in.txt
```

```
Hello World *17:25:01*0 *10.10.2000*
*00:00:00*
99:99:99
0:00:0
*12:12:12*
*14:13:14*
*17:17:17*
*21.12.2002*
00:
00:00 *00.00.9999*

10.
01.20
00
*15.01.2022*
```

При указании выходного файла, весь вывод записывается в указанный файл

```
~ /C_Workspace/prg3vypN3150 ./prg3vypN3150 in.txt out.md
```

При возникновении ошибок программа выводит информацию о них в консоль:

```
~ /C_Workspace/prg3vypN3150 ./prg3vypN3150 -g
./prg3vypN3150: invalid option -- 'g'
~ /C_Workspace/prg3vypN3150 ./prg3vypN3150 -b=A4
Ошибка: 'A4' не является целым положительным числом.
~ /C_Workspace/prg3vypN3150 ./prg3vypN3150 in1.txt
Ошибка: не удалось открыть файл in1.txt.
~ /C_Workspace/prg3vypN3150 ./prg3vypN3150 -v -n -c -b=1 -e=10 in.txt out.md arg
Ошибка: слишком много аргументов.
~ /C_Workspace/prg3vypN3150
```

Исходный текст программы с комментариями

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

#include <getopt.h>
#include <unistd.h>

// Подсчёт количества символов \n
int count_newlines(const char *str) {
    int count = 0;
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] == '\n') {
            count++;
        }
    }
    return count;
}

// Пропуск символов перехода на следующую строку
int skip_newlines(const char *str, int *pos) {
    while (str[*pos] == '\n') {
        (*pos)++;
    }
    return *pos;
}

// Поиск времени в формате HH:MM:SS с учетом разрывов строк
int find_time_pattern(const char *str, int *start, int *end, int n_flag) {
    int pos = *start;
    int original_pos = pos;

    // Проверка часов (HH)
    if (!(str[pos] >= '0' && str[pos] <= '2')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (!((str[pos-1] == '2') && (str[pos] >= '0' && str[pos] <= '3')) || ((str[pos-1] >= '0' && str[pos-1] <= '1') && (str[pos] >= '0' && str[pos] <= '9')))) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (str[pos] != ':') return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);

    // Проверка минут (MM)
    if (!(str[pos] >= '0' && str[pos] <= '5')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (!(str[pos] >= '0' && str[pos] <= '9')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (str[pos] != ':') return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);

    // Проверка секунд (SS)
    if (!(str[pos] >= '0' && str[pos] <= '5')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (!(str[pos] >= '0' && str[pos] <= '9')) return 0;
    pos++;

    *start = original_pos;
    *end = pos;
    return 1;
}

// Поиск даты в формате dd.mm.yyyy с учетом разрывов строк
int find_date_pattern(const char *str, int *start, int *end, int n_flag) {
    int pos = *start;
```

```

    int original_pos = pos;

    // Проверка дня (dd)
    if (!(str[pos] >= '0' && str[pos] <= '3')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (!(str[pos-1] == '3' && str[pos] >= '0' && str[pos] <= '1') || (str[pos-1] >= '0' &&
str[pos-1] <= '2' && str[pos] >= '0' && str[pos] <= '9')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (str[pos] != '.') return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);

    // Проверка месяца (mm)
    if (!(str[pos] >= '0' && str[pos] <= '1')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (!(str[pos-1] == '1' && str[pos] >= '0' && str[pos] <= '2') || (str[pos-1] == '0' &&
str[pos] >= '0' && str[pos] <= '9')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (str[pos] != '.') return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);

    // Проверка года (yyyy)
    if (!(str[pos] >= '0' && str[pos] <= '9')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (!(str[pos] >= '0' && str[pos] <= '9')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (!(str[pos] >= '0' && str[pos] <= '9')) return 0;
    pos++;
    if (!n_flag) skip_newlines(str, &pos);
    if (!(str[pos] >= '0' && str[pos] <= '9')) return 0;
    pos++;

    *start = original_pos;
    *end = pos;
    return 1;
}

// Выделение найденных подстрок *Курсивом*
char* mark_times_and_dates(const char* input, int n_flag, int b_value, int e_value, int c_flag) {
    int input_len = strlen(input);
    char* output = malloc(input_len * 3 + 1); // Увеличиваем размер буфера для учета ANSI кодов
    char* out_ptr = output;
    int i = 0;
    int count_n = 0;

    // Пропуск строк до b_value
    while (i < input_len && count_n < b_value - 1) {
        if (input[i] == '\n') {
            count_n++;
        }
        *out_ptr++ = input[i++];
    }

    // Обработка строк от b_value до e_value
    while (i < input_len && count_n < e_value) {
        int start = i;
        int end = i;

        if (find_time_pattern(input, &start, &end, n_flag) || find_date_pattern(input, &start,
&end, n_flag)) {
            // Проверка, что время или дата полностью находится в пределах строк
            int line_start = start;
            int line_end = end;
            while (line_start > 0 && input[line_start - 1] != '\n') line_start--;
            while (line_end < input_len && input[line_end] != '\n') line_end++;

```

```

int line_count_start = 0;
int line_count_end = 0;
for (int j = 0; j < line_start; j++) {
    if (input[j] == '\n') line_count_start++;
}
for (int j = 0; j < line_end; j++) {
    if (input[j] == '\n') line_count_end++;
}

if (line_count_start >= b_value - 1 && line_count_end < e_value) {
    if (c_flag) {
        // Добавляем ANSI код для зеленого цвета
        *out_ptr++ = '\033';
        *out_ptr++ = '[';
        *out_ptr++ = '3';
        *out_ptr++ = '2';
        *out_ptr++ = 'm';
    } else {
        *out_ptr++ = '*';
    }

    memcpy(out_ptr, input + start, end - start);
    out_ptr += end - start;

    if (c_flag) {
        // Возвращаем цвет к стандартному
        *out_ptr++ = '\033';
        *out_ptr++ = '[';
        *out_ptr++ = '0';
        *out_ptr++ = 'm';
    } else {
        *out_ptr++ = '*';
    }
} else {
    memcpy(out_ptr, input + start, end - start);
    out_ptr += end - start;
}
i = end;
} else {
    if (input[i] == '\n') {
        count_n++;
    }
    *out_ptr++ = input[i++];
}
}

// Копирование оставшейся части строки после e_value без обработки
while (i < input_len) {
    *out_ptr++ = input[i++];
}

*out_ptr = '\0';
return output;
}

int main(int argc, char *argv[]) {
    // Проверка запуска с переменной среды, включающей отладочный вывод.
    // Пример запуска с установкой переменной LAB3DEBUG в 1:
    // $ LAB3DEBUG=1 ./prg3abcNXXXXX test.txt
    char *DEBUG = getenv("LAB3DEBUG");
    if (DEBUG) {
        fprintf(stderr, "Включен вывод отладочных сообщений\n");
    }

    // Автор
    char *author = "Пахомов Владимир Юрьевич, гр. N3150\nВариант: 8-1-1-2\n";

    // Переменные опций
    int opt;

```



```

int c_flag = 0; // выделение цветом
int v_flag = 0; // информация об авторе
int n_flag = 0; // отключение \n
int b_value = 1; // начальная строка
int e_value = 0; // конечная строка

// Переменные ввода/вывода
FILE *input_file = NULL;
FILE *output_file = NULL;
size_t input_size = 0;
char *input_str = NULL;
char *input_filename = NULL;
char *output_filename = NULL;

char *endptr;
char *value;
// Обрабатываем опции командной строки
while ((opt = getopt(argc, argv, "vcnb:e:")) != -1) {
    if (opt == 'v') v_flag = 1;
    else if (opt == 'c') c_flag = 1;
    else if (opt == 'n') n_flag = 1;
    else if (opt == 'b') {
        value = optarg;
        if (value[0] == '=') {
            value++;
        }
        b_value = strtol(value, &endptr, 10);
        if (endptr == value || *endptr != '\0' || b_value < 0) {
            fprintf(stderr, "Ошибка: '%s' не является целым положительным числом.\n", value);
            return EXIT_FAILURE;
        }
    }
    else if (opt == 'e') {
        value = optarg;
        if (value[0] == '=') {
            value++;
        }
        e_value = strtol(value, &endptr, 10);
        if (endptr == value || *endptr != '\0') {
            fprintf(stderr, "Ошибка: '%s' не является целым положительным числом.\n", value);
            return EXIT_FAILURE;
        }
    }
    else {
        return EXIT_FAILURE;
    }
}

// Обработка аргументов ввода/вывода
int args_left = argc - optind;
if (args_left > 2) {
    fprintf(stderr, "Ошибка: слишком много аргументов.\n");
    return EXIT_FAILURE;
}
else if (args_left == 2) {
    input_filename = argv[optind];
    output_filename = argv[optind + 1];
}
else if (args_left == 1) {
    input_filename = argv[optind];
}

// Открываем входной файл, если указан
if (input_filename != NULL) {
    input_file = fopen(input_filename, "r");
    if (input_file == NULL) {
        fprintf(stderr, "Ошибка: не удалось открыть файл %s.\n", input_filename);
        return EXIT_FAILURE;
    }
}
else {
    input_file = stdin;
}

// Открываем выходной файл, если указан
if (output_filename != NULL) {

```

```

        output_file = fopen(output_filename, "w");
        if (output_file == NULL) {
            fprintf(stderr, "Ошибка: не удалось открыть файл %s.\n", output_filename);
            return EXIT_FAILURE;
        }
    } else {
        output_file = stdout;
    }

    // Вывод информации об авторе
    if (v_flag) {
        fprintf(output_file, "%s\n", author);
        return EXIT_SUCCESS;
    }

    // Чтение входных данных
    char buffer[1024];
    while (fgets(buffer, sizeof(buffer), input_file)) {
        size_t len = strlen(buffer);
        input_str = (char*) realloc(input_str, input_size + len + 1);
        if (input_str == NULL) {
            fprintf(stderr, "Ошибка: не удалось выделить память.\n");
            fclose(input_file);
            fclose(output_file);
            free(input_str);
            return EXIT_FAILURE;
        }
        strcpy(input_str + input_size, buffer);
        input_size += len;
    }

    // Обработка конца строки ввода
    if (ferror(input_file)) {
        perror("Ошибка чтения ввода.");
        fclose(input_file);
        fclose(output_file);
        free(input_str);
        return EXIT_FAILURE;
    }

    // Если -e не указан (то есть не изменился после обработки опций), то берем все строки для
    преобразования
    if (e_value == 0) e_value = count_newlines(input_str) + 1;

    // Поиск подстрок и вывод результата
    char *result = mark_times_and_dates(input_str, n_flag, b_value, e_value, c_flag);
    fprintf(output_file, "%s\n", result);

    // Освобождение памяти и закрытие потоков
    free(input_str);
    fclose(input_file);
    fclose(output_file);

    return EXIT_SUCCESS;
}

// 8-1-1-2
// 8 Дата и время В формате HH:MM:SS dd.mm.yyyy Примеры: 12:24:01 01.02.2001, 00:00:00
// 31.12.2023
// 1 -b=M - номер начальной строки (нумерация с 1), -e=N - номер конечной строки (включая
// её)
// 1 Markdown Курсив *Пример*
// 2 Выделение Зеленый 32

```